

凸性与约束的几何 实验指南

第十六讲 先检查可行性 再比较距离

本实验要回答：同一个建议点受到不同限制时，最近允许点怎样改变；怎样用可行残差、距离和内积证书区分正确投影与只是可行的结果。读完正文的一阶判据与投影证明后再做实验，先写出预测，再运行代码，最后解释偏差。预计需一次完整学习时段，可按五个阶段分两次完成。

全部输入为固定、原创、无量纲教学数据。计算仅使用小规模CPU数组，没有训练模型、网络请求、真实用户数据、GPU或付费接口。实验不能说明现实模型的预测性能，也不能把几个成功数值例子当成全域凸性证明。

一 文件 环境与运行顺序

将 `experiment.py`、`experiment.ipynb` 与 `data` 目录放在同一层。脚本用自身所在目录定位数据，因而命令行的当前工作目录不必等于单元目录。Notebook默认从本单元目录打开；若导入失败，先检查打开位置，不要复制粘贴一份不同版本的核心函数。

核心脚本只依赖NumPy；Notebook另用Matplotlib、IPython与Jupyter。实际制作环境为Python 3.12.14、NumPy 2.3.5、Matplotlib 3.10.8。建议环境文件用于复现依赖，不代表已经在读者的Anaconda、操作系统或浏览器界面测试过。

在单元目录中运行：

```
python experiment.py
jupyter lab experiment.ipynb
```

也可先用提供的 `environment.yml` 建立环境，再运行以上命令。Notebook应重启内核后按顺序执行全部单元；不是只运行最后一个已经保留答案的单元。若想检查脚本的公共入口，在新终端进程再次运行 `python experiment.py`，比较两个报告中的固定数值。

脚本会重建 `outputs/experiment-report.json`、`projection-results.csv`、`intersection-comparison.csv` 和 `grid-diagnostics.csv`。这是可再生输出目录，不是额外原始数据。JSON汇总手算、边界检查和有限候选检查；CSV便于逐行比较输入、投影、平方距离与可行残差。

二 理解两个输入文件

`data/points2d.csv` 有四行二维点。`main=(3,2)`贯穿正文；`inside=(0.5,0.5)`测试已经合法的点；`negative=(-1,3)`测试下界和上界同时越界；`origin=(0,0)`测试原点与球中心分支。字段`point_id`是标签，`z1`、`z2`是坐标，不是样本特征和预测目标。

`data/affine_system.csv` 的两行分别代表 $z_1+z_2=1$ 、 $z_2+z_3=0$ ， a_1 至 a_3 是系数， rhs 是右端常数。它对应三维建议点(3,2,1)，不能把二维点表直接输入这个系统。行标识E1、E2用于检查文件没有被误换。

加载器故意要求这两份固定数据与课堂已知答案一致。如果想增加新点，保留原文件，另写探索代码；不要修改CSV后仍引用本讲的16组验收结果。这里采用严格固定数据，是为了让读者能确定地复核正文，不是通用数据导入工具的设计建议。

三 运行前先手算五个检查点

在看到程序答案前，写下以下结果及理由。

1. $v=(3,2)$ 到盒子 $[0,2] \times [0,1]$ 的投影为(2,1)，平方距离为2。
2. v 到原点半径2圆盘的投影为 $(6/\sqrt{13}, 4/\sqrt{13})$ ，平方距离为 $17-4\sqrt{13}$ 。
3. v 到直线 $z_1+z_2=1$ 的投影为(1,0)，平方距离为8。
4. 输入1、2、4，权重 $1/2$ 、 $1/4$ 、 $1/4$ ，先平均再平方为4，先平方再平均为 $11/2$ 。
5. 三维仿射投影是 $(4/3, -1/3, 1/3)$ ，平方距离为 $26/3$ 。

每一项都写出一个独立理由，例如区间端点下界、三角不等式、残差正交或分数算术。不能只把期待结果塞回同一个程序，再称它是独立验证。

四 第一阶段 让定义进入数组契约

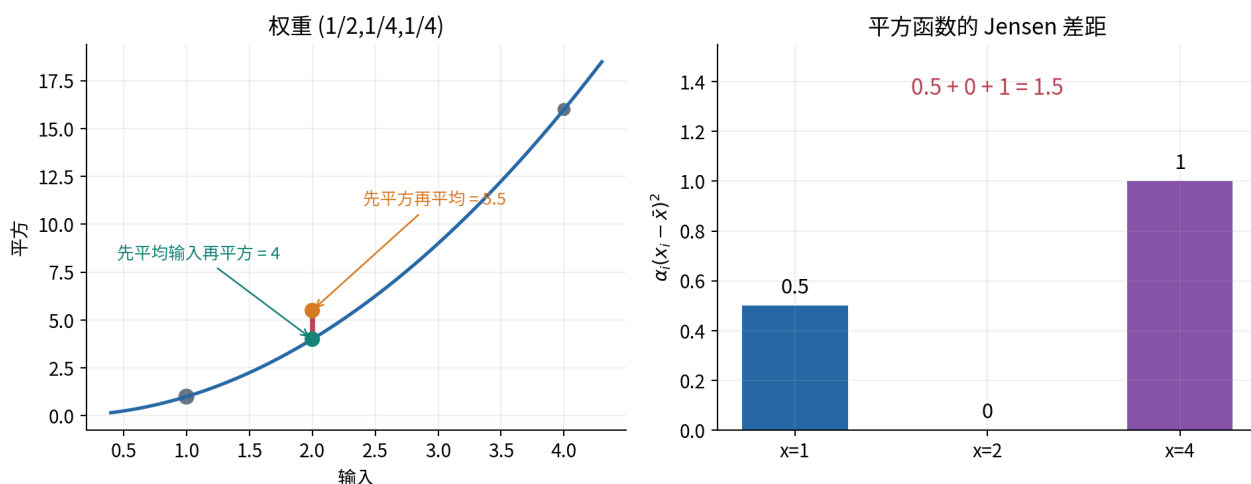
`convex_combination(points, weights)` 接收 m 个 n 维点组成的 (m,n) 矩阵，以及长度 m 的一维权重。程序先检查原始类型，再转成`float64`，拒绝布尔、字符串、复数、`object`数组、非有限值、空集合和错误形状。

运行三角形顶点(0,0)、(2,0)、(0,2)及权重(0.5,0.25,0.25)，应得到(0.5,0.5)。然后试权重(1,1,-1)：程序应拒绝，即使其和仍为1。手动算出(2,-2)，并解释它为什么只是仿射组合。

权重和检查采用绝对容差 $1e-12$ ；接受后不重新归一化，保存原来的权重和。容差是数值输入约定，定理仍要求精确和为1。不要把容差调到很大后说数学条件“仍然满足”。本讲已知答案使用可精确表示的二进制分数权重，避免这项近似影响核心例子。

`jensen_diagnostic` 比较 $f(\text{平均点})$ 与函数值加权平均。它先检查平均点有限，再调用函数，所有回调返回值必须为有限实标量。它不知道用户函数的整个定义域，也无法通过一个样本判断全域凸性；返回值中的 `holds_within_tolerance` 只报告本次有限比较。

对于 $f(x)=x^2$ 应得到差距1.5。把函数换成 x^4-x^2 、输入改成 $\pm 1/\sqrt{2}$ 、权重等分，会得到差距-0.25，直接出现违反Jensen的反例。一个失败例足以否定凸性；若有限点全部通过，只能说没有在这些点找到反例。



平方函数的 Jensen 差距由加权离均差解释，程序应报告4、5.5和1.5。

五 第二阶段 逐个集合实现投影

首先查看区间接口 `project_interval`。输入不小于下界且不大于上界时保持原值；越界时返回相应端点；下界等于上界合法，逆序边界非法。对 $[0,1]$ 测试-2、0、0.4、1、3，期望分别是0、0、0.4、1、1。

盒子接口要求 v 、`lower`、`upper`全都为同长度一维向量。这里不允许广播标量或 $(1,n)$ 行矩阵。NumPy的`clip`本身不会替你检查下界不大于上界，因此封装必须主动检查。对四个固定建议点，分别记录每个坐标变化，不要只看总距离。

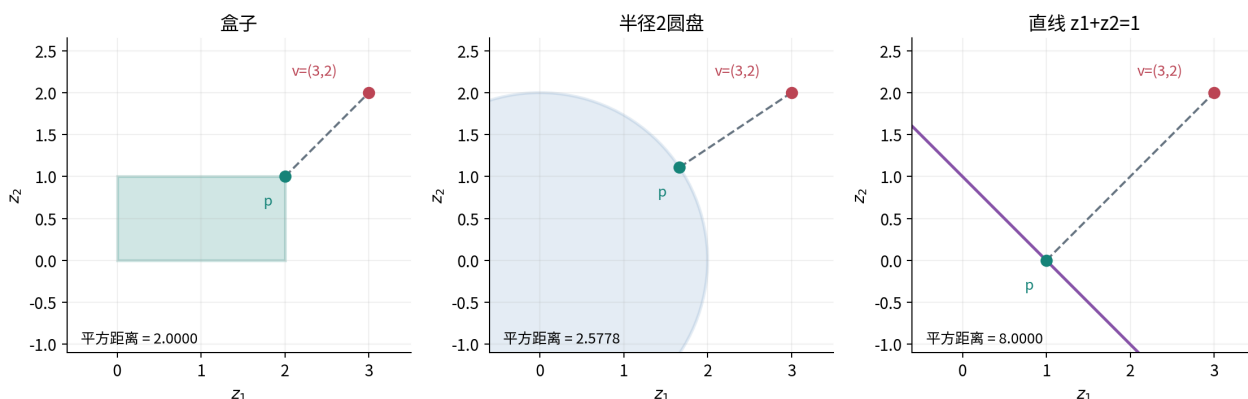
球接口先检查非负半径。 $R=0$ 直接返回中心； v 在球内则原样返回；球外才使用径向缩放。对 $\text{inside}=(0.5,0.5)$ ，投影仍为自己，不应被推到半径2的球面。对平移球 $c=(1,-1)$ 、 $R=2$ 、 $v=(4,3)$ ，期望 $(11/5, 3/5)$ ，平方距离9。

超平面接口要求法向量非零，且形状与 v 一致。半空间接口还要判断原点是否已经符合不等式，合法时不动。对 $w=(0,0)$ 和 $z_1+z_2 \leq 1$ ，投影是 w ；对同一 w 和等式 $z_1+z_2=1$ ，投影是 $(1/2, 1/2)$ 。两种任务只有一个符号不同，答案却不同。

对每个输出 p 执行三个分开的检查：

- 可行性：逐坐标界限、球内范数、等式残差或不等式正违反量
- 距离：计算真实平方距离，必要时与解析公式比较
- 最优性依据：引用对应集合的解析证书；有限候选仅用于排查实现错误

另检查幂等性 $P_C(P_C(v)) = P_C(v)$ 。合法投影再次输入同一映射应不变，但幂等性本身不足以证明最近性，很多并非欧氏投影的映射也可能幂等。



同一个 v 在不同集合上产生不同投影。比较平方距离之前先确认是在回答哪个约束问题。

六 第三阶段 检查线性约束与允许方向

先手算直线例。若 $v=(3,2)$ 、 $a=(1,1)$ 、 $b=1$ ，则修正倍数为 $(5-1)/2=2$ ，输出 $(1,0)$ 。残差 $(2,2)$ 与所有 $(t, -t)$ 正交。改成区间 $[0,1]$ 上的一维目标 $(x-3)^2/2$ ，最优点1的导数为-2；画出负梯度指向区间外的箭头，解释为什么边界最优不要求零梯度。

三维系统的A形状为 $(2,3)$ ，b形状为 $(2,)$ ，v形状为 $(3,)$ 。project_affine 用 `lstsq(A,b-A@v)` 求最小范数修正。不要显式计算矩阵逆，也不要欠定系统的空残差返回数组当成零误差。程序自己计算 $A@p-b$ ，同时核验数值秩。

本接口要求 $1 \leq m \leq n$ 并且A数值满行秩。它会拒绝冗余行和矛盾行；这是清楚限制后的接口范围。一组一致的冗余等式在数学上仍能有唯一投影，只是不属于本实现接受的输入。数值秩还受浮点阈值影响，不能代替符号证明。

接着沿 $d=(1,-1,1)$ 采样 $p+td$ ，令 t 取-2、-1、0、1、2。每个采样点都应满足两条等式，平方距离应等于 $26/3+3t^2$ 。 $t=0$ 是唯一最小，左右两边的增加对称。这从图和数值上展示了正交性为什么使一次项消失。

line_samples 在执行任何回调之前，先构造并检查所有采样点有限。如果基点、方向和步长虽各自有限，其乘加结果已经溢出，实验应立即拒绝，回调调用次数必须为0。这防止一段“恒返回0”的用户函数掩盖非法采样点。

七 第四阶段 让错误的顺序投影暴露出来

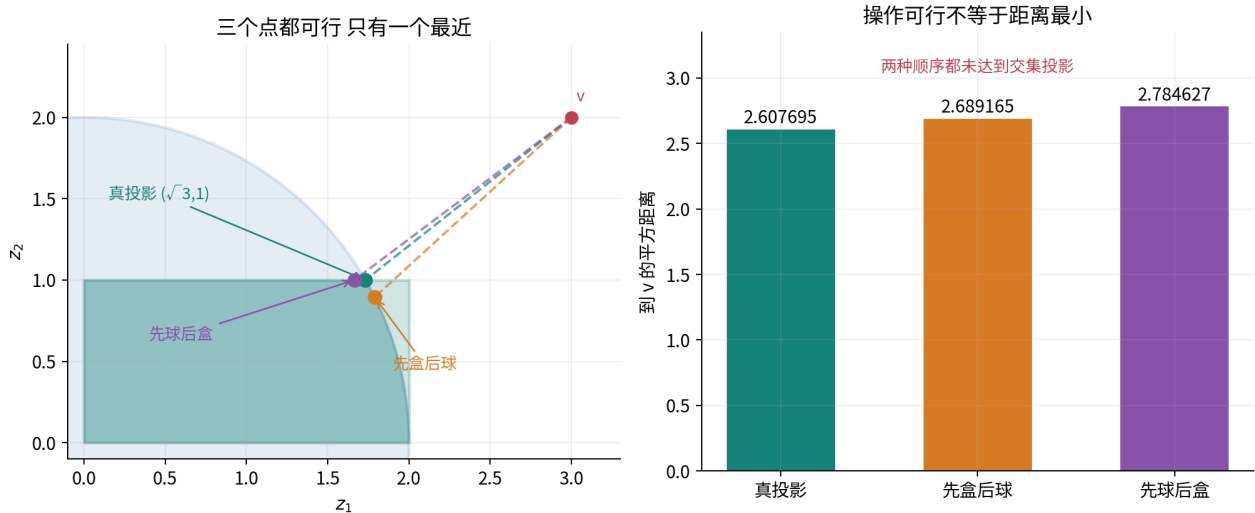
现在共同要求盒子和圆盘限制。先写下猜测：先裁剪再缩放会不会恰好是交集投影？反过来顺序是否一样？然后执行两种操作并比较。

先盒后球得到 $(4/\sqrt{5}, 2/\sqrt{5})$ ，先球后盒得到 $(6/\sqrt{13}, 1)$ 。它们都符合两个限制，却分别给出平方距离约2.689165与2.784627。真正投影 $(\sqrt{3}, 1)$ 的平方距离约2.607695，更小。

不能只看柱状图就宣称真投影是全域最优。请复述正文的证书： $v-p=(\sqrt{3}-1)p+(2-\sqrt{3})e_2$ 。对于交集内任意 z ，圆盘给出 $p^T(z-p) \leq 0$ ，盒子的第二坐标上界给出 $e_2^T(z-p) \leq 0$ 。非负加权后便是投影证书。

再以 p 作为反例点，检查两个错误候选 q 的内积 $(v-q)^T(p-q)$ 。输出约0.047921和0.090775，均为正，说明这两个候选没有通过必要的投影证书。它们是“可行但非最优”的直接教材，不是浮点误差导致的轻微排序波动。

有限网格的用途也要写清楚。脚本在盒子内构造 101×101 个候选，筛出同时在圆盘内的点，比较最小网格距离。网格通常不含精确的 $(\sqrt{3}, 1)$ ，所以网格最小距离可能略大于解析最小距离。这个正差距来自离散化，不能倒过来否定解析答案。无限集合上的最优性来自证书，不来自网格数量。



两个顺序都给可行点，却都没有达到交集真投影的距离。注意左图重叠区域与右图距离排序。

八 第五阶段 有意触发失败并解释边界

完整脚本包含71个边界案例：61项应拒绝，10项应接受。不要以“没有报错”为唯一目标；对不满足数学契约的输入，清楚拒绝本身就是正确行为。

特别逐项检查：混合布尔/字符串、object数组、二维行列向量冒充一维向量、上下界逆序、负半径、零法向量、空点集、负权重、权重和错误、仿射冗余行与矛盾行。对合法边界检查单点区间、 $R=0$ 、球中心、球内部、半空间边界、零权重和零方向。

有限性还需要检查中间值和输出。下列输入表面有限，却可能无法可靠处理： $1e200$ 向量的平方范数溢出；两个约为 $1e308$ 的数相减溢出；约 $1e-250$ 的向量平方范数下溢成0。此实现会拒绝这些尺度，不声称可以自动准确缩放所有极端问题。不要关闭浮点警告后把inf或NaN当作有效投影。

回调也受同一规则约束：返回inf、NaN、布尔值或数组都被拒绝，函数值必须为有限实标量。中间采样点先于回调接受检查，避免非法点被忽略。Notebook保留了这些故意失败的可读摘要；预期异常是被测试代码捕获的，不应成为Notebook的未处理错误输出。

九 如何阅读验收记录

`projection-results.csv` 共16行，是四个固定点分别投影到盒子、球、超平面、半空间的结果。
`objective_half_squared_distance` 应恰为 `squared_distance` 的一半。所有可行违反量应在 $1e-10$ 范围内；具体残差的数量级不等同于精确为零的证明。

`intersection-comparison.csv` 保存三个候选点、平方距离以及以真投影作测试点时的证书内积。
`grid-diagnostics.csv` 保存每类集合的候选数、最大证书内积和网格最小距离相对解析答案的差值。
 候选集合在调用证书检查前已按对应规则构造，函数名本身不会替你证明点可行。

JSON还包括四种同集合投影的非扩张抽查，每种比较四个输入的16个有序对。得到非正的“投影后距离平方减去原距离平方”与定理一致，但64个有限比较不是一般定理证明。若两次使用不同集合，正文推导中交换候选点的步骤会失效，不能套用这项结论。

本讲脚本已在新Python进程运行并重跑；Notebook已在新进程的真实ipykernel InProcessKernel中清空状态、顺序执行并保存PNG图像输出。因为构建环境不能使用socket传输，该核验未覆盖外进程socket、Jupyter浏览器界面和读者本机Anaconda安装。保留输出方便阅读，仍应按顺序亲自运行以理解计算过程。

十 提交前的解释题

不看答案，用自己的话回答：为什么凸集必须检查任意线段？为什么严格凸只能先保证“至多一个”最优点？为什么边界上的负梯度可能不能使用？为什么合法点再次投影不变？为什么有限网格没有超过解析最优点仍不算证明？为什么先满足一个限制、再满足另一个限制，没有自动解决交集最近点问题？

提交内容应包含五项手算、16行投影表、三维直线的距离关系、交集真投影的全域证书、至少三项拒绝输入的理由。最后分别写一句“理论证明了什么”和“程序只检查了什么”。能保持这两句话的边界，比仅仅得到一串相同小数更重要。

参考接口

NumPy接口事实以官方版本文档为准：`clip` 与 `lstsq`。数学定义、存在性条件与证明来源见正文末尾及 `source-checks.json`；本实验数据、例子、图形和代码独立编写。